VIPS · Pitampura · B.Tech 1st sem

SkillSwap

User Manual & Handover

Campus knowledge network — leftover items (free) + 25-minute Google Meet sessions (free) + student/teacher feed.

Live site: https://skillswap-3ca53.web.app

This document is for handover: how to use the site, how it was built, where data is saved, and how the logic works.

1. Login (ready accounts)

Role	Email	Password	What it is for
Student	`ansh@vips.edu`	`campus123`	Normal campus user
Admin	`admin@vips.edu`	`campus123`	Moderate posts, items, sessions
Teacher	create your own	min 6 characters	Faculty posts and Meet hours

Open https://skillswap-3ca53.web.app → **Log in**.

New account: Create account → name, email, password twice → Student or Teacher → programme → **Create account**. Admin is not in the dropdown. Only `admin@vips.edu` becomes Admin.

Password must be at least 6 characters. Same email works on any browser (data is online).

2. How to use (every role)

After login, the top bar is: **Feed · Items · Knowledge · People · About · Inbox · (your name) · Post · Log out**. Admin also sees **Moderate**.

> There are **no public comments**. To ask someone, open their profile and send an Inbox message.

2.1 Student (`ansh@vips.edu`)

1. **Feed** — campus posts. Type in the box, pick Share knowledge / Need something / Campus update, click **Post**. Mark **Helpful**. Filters: All, Following, Faculty, Knowledge, Needs, Updates.
2. **Items** — leftover books, drafter, notes. Open a card → **Request free item**. Owner gets it in Inbox. No payment.
3. **Knowledge** — 25-min Google Meet sessions. Open one → **Join Meet**.
4. **People** — follow classmates and teachers. Profile → Follow or private message.
5. **Post** (green) — list a leftover item (upload your photo) or host a free Meet (paste a Meet link).

6. **Inbox** — item requests, session joins, private messages.
7. **Your name** — your profile, listings, sessions you joined. You can delete your own posts/listings.

Demo path: Items → request mini-drafter. Knowledge → join GitHub session. Feed → post a need.

2.2 Teacher (create account, role = Teacher)

Same screens as student, used for class:

- Feed: lab format, extra class, notice.
- **Post** → knowledge session: title, what you will cover, time, **Google Meet link**.
- Students follow you. **Faculty** filter on the feed shows only teacher posts.
- Replies come in **Inbox**, not as public comments.
- You can delete your own posts/listings. You cannot remove other people's content (that is Admin).

2.3 Admin (`admin@vips.edu`)

Same as a logged-in user, plus **Moderate**:

1. Open **Moderate**.
2. Three lists: Feed posts, Items, Meet sessions.
3. **Remove** → confirm. That listing disappears for everyone.
4. On the feed, **Remove** is also on every post. On an item/session page: Remove listing / Remove session.

Use this for spam or a Meet that should not stay public.

3. How the website was built

Layer	What we used
UI	React 19 (JavaScript, not TypeScript)
Build / local server	Vite 8
Pages	React Router DOM 7
Icons	Lucide
Login + database	Google Firebase (Spark / free)
Hosting	Firebase Hosting

Important folders

```
d:\ProjectAnsh
├── src\
│   ├── App.jsx          ← login gate + top nav + routes
│   ├── context.jsx      ← all business logic (signup, post, like, follow, delete)
│   ├── data.js          ← seed people, posts, items, sessions, admin email
│   ├── index.css        ← UI
│   ├── pages\           ← Feed, Items, Knowledge, People, Inbox, Post, Profile, Admin,
About
│   ├── components\      ← cards (item, session, feed post)
│   └── lib\
│       ├── firebase.js  ← connects to Firebase using .env keys
│       ├── cloud.js     ← read/write Firestore
│       └── localAuth.js ← backup login if .env is missing (this computer only)
├── public\              ← VIPS logo + campus photos
├── firestore.rules      ← who can read/write the database
├── firebase.json        ← hosting + rules
├── .env                 ← Firebase web keys (not committed to Git)
└── dist\                ← production build uploaded to Hosting
```

Local run:

```
npm install
npm run dev
```

Open the URL Vite prints (often http://localhost:5173 or 5175).

Live deploy (already done): `npm run build` then `npx firebase-tools deploy --only hosting`. Firebase project id: `skillswap-3ca53`.

4. Where data is saved (logic)

Two Google services, one job each:

What	Where it is stored	Why
Email + password	**Firebase Authentication**	Passwords are hashed by Google. They never go into our campus database.
Profile, posts, items, sessions, inbox, follow list	**Cloud Firestore**	Documents the website reads and writes.
Website files (HTML/CSS/JS, logo, photos)	**Firebase Hosting**	The live URL.

Firestore collections (online database tables):

Collection	What is inside
`users`	Name, email, role (Student / Teacher / Admin), branch, bio, following
`posts`	Feed text, type, author, likes
`resources`	Leftover items, photo, status (Available / Requested)
`skills`	Meet sessions, Meet link, slots
`inbox`	Private messages, item requests, session joins
`bookings`	Who joined which session
`meta/seed`	Marker so sample campus data is loaded only once

Security rule: signed-in users can read/write. Guests see only the login page. File: `firestore.rules` → `allow read, write: if request.auth != null`.

Admin rule in code (not a Google role): if email is `admin@vips.edu`, role becomes Admin (`src/data.js`). Signup dropdown cannot pick Admin.

Photos of leftover items: the user uploads a file; the browser resizes it and stores it as an image string on the item document in Firestore. Not a separate Storage bucket.

If `.env` is missing (local demo only): accounts go to browser localStorage (`skillswap-accounts-v1`, hashed password). The live site always uses Firebase.

Footer **Accounts online · Firebase** means the cloud path is on.

5. How it works (step by step)

```
Browser (React)
    │
    ├─ Login / Signup ──► Firebase Authentication  (email + password)
    │
    └─ After login ─────► Cloud Firestore listeners (live updates)
                              users, posts, resources, skills, inbox, bookings
```

1. **Open site** — App.jsx waits until Firebase says who is logged in. No user → login/signup screen. User → campus layout.
2. **Signup** — Firebase creates the auth user. Then a users/{uid} document is written (name, role, branch). Password stays in Authentication only.
3. **Login** — Firebase checks email/password. On success, onAuthStateChanged sets the session. listenCampus attaches live listeners; if a classmate posts, your feed updates without refresh.
4. **First login ever** — ensureSeed() copies sample VIPS data (faculty, drafter, GitHub session) into Firestore once.
5. **Post on feed** — addPost writes to posts. Everyone signed in sees it.
6. **Helpful** — your user id is added/removed on that post's likes array.

7. **Follow** — your id is added to your following array on users.
8. **List an item / session** — new document in resources or skills, plus an automatic feed post.
9. **Request item / join Meet** — listing status or booking is saved; an inbox row is created; Meet opens in a new tab.
10. **Admin Remove** — deleteDoc on that post/item/session. Listeners drop it from every open browser.

Roles: Student / Teacher are chosen at signup. Admin is forced when email is `admin@vips.edu`. Moderate page (`/admin`) redirects home if you are not admin.

6. What this is not

- Not OLX (items are free, campus only).
- Not a paid mentorship app (Meet is free).
- Not Facebook (no public comment threads).
- Not a full moderator product: one admin email, client-side role check, any signed-in user can technically write in Firestore. Fine for a 1st-sem prototype; tighten rules before a real campus launch.

7. Handover checklist

- Open https://skillswap-3ca53.web.app
- Student login `ansh@vips.edu` / `campus123` — feed, item request, join Meet
- Log out
- Admin login `admin@vips.edu` / `campus123` — Moderate → Remove
- Create a Teacher account and post a Meet session
- Firebase console (same Google account that owns the project): Authentication (users), Firestore (data), Hosting (site)
- Keep `.env` private. It is gitignored. Without it, npm run dev falls back to this-computer-only login.

Firebase console: https://console.firebase.google.com/project/skillswap-3ca53/overview

Project owner Google account: the one used to create SkillSwap (`anshbutola96@gmail.com`).